Technical Report

Elevator Control System:

Data Structures & Algorithms Project

Team 4

Instructor : Eng.Khalid Alkhulayfi

December 9, 2025

1. Problem Description and Objectives

1.1 Problem Statement

Elevator scheduling is a critical component in building management systems, directly impacting user satisfaction, energy efficiency, and overall system performance. The challenge lies in determining the optimal order to service elevator requests while minimizing passenger wait times and total travel distance. This project addresses the fundamental question: which scheduling algorithm provides better performance in a multi-floor building environment?

The problem involves managing a queue of elevator requests, where each request consists of a starting floor and a destination floor. The elevator must service all requests while adhering to physical constraints such as sequential floor traversal and realistic timing intervals.

1.2 Project Objectives

This project aims to achieve the following objectives:

- **Implement and compare two distinct elevator scheduling algorithms: First-Come-First-Served (FCFS) and LOOK**
- **Develop a real-time simulation system with realistic timing constraints (0.4s request generation, 0.2s floor movement)**
- **Measure and analyze comprehensive Key Performance Indicators (KPIs) including execution time, memory usage, throughput, wait time, and CPU utilization**
- **Provide visual comparison through a graphical user interface to demonstrate algorithm behavior in real-time**
- **Analyze trade-offs between fairness (FCFS) and efficiency (LOOK) to provide recommendations for different use cases**

1.3 Scope and Constraints

The project operates within the following constraints:

- Single elevator system (4 floors: A, B, C, D)
- Sequential floor traversal (must pass through intermediate floors)
- Random request generation with configurable sample size (n)
- Fixed timing intervals for realistic simulation
- Python implementation with Tkinter GUI framework

2. System Design and Flowcharts

2.1 System Architecture

The system follows a modular architecture with clear separation of concerns. The main components include:

- **GUI Layer (Tkinter):** Handles user input, visualization, and real-time display updates
- **Simulation Controller:** Manages request generation and algorithm execution timing
- **Algorithm Modules (FCFS & LOOK):** Implement scheduling logic and elevator movement
- **Metrics Tracker:** Collects and calculates performance indicators
- **Data Structures:** Request queue, state tracking, and metric storage

2.2 System Workflow

Initialization Phase:

- User inputs desired number of requests (n) through GUI
- System validates input and initializes data structures
- Baseline memory is captured for net usage calculation
- Both elevators positioned at floor A (initial state)

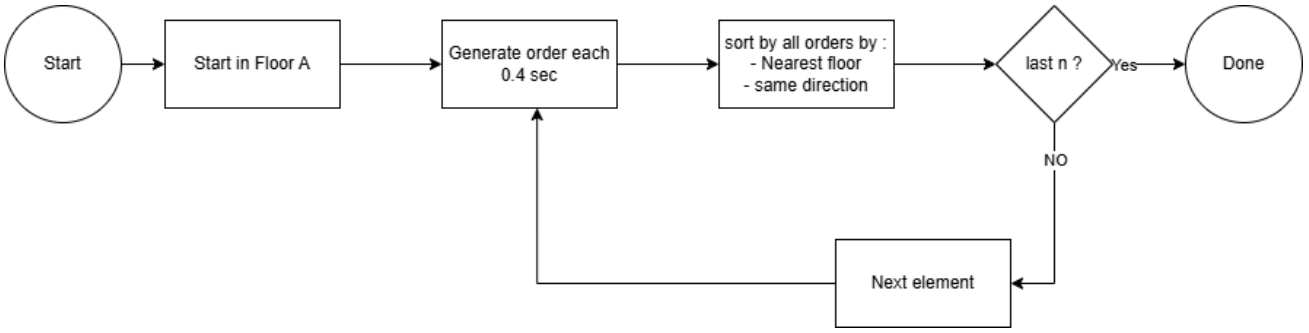
Execution Phase (Dual-Loop Architecture):

- **Generation Loop (400ms interval):** Creates one random request every 0.4 seconds until n requests are generated
- **Processing Loop (200ms interval):** Executes one step of each algorithm (moves elevator one floor) every 0.2 seconds

Termination Phase:

- All requests generated (n reached)
- Both algorithms complete processing all requests
- Final metrics calculated and displayed
- System ready for next simulation run

2.3 Data Flow Diagram



3. Explanation of Selected Data Structures and Algorithms

3.1 Data Structures

3.1.1 Request Object

The Request dataclass serves as the fundamental unit of work in the system:

- **start (int):** Starting floor (1-4, mapped from A-D)
- **dest (int):** Destination floor (1-4, different from start)
- **gen_time (float):** Timestamp of request creation for wait time calculation
- **id (int):** Unique identifier for tracking purposes

3.1.2 State Tracking Lists

Each algorithm maintains parallel state lists to track request progress:

- **requests_numeric (List[Request]):** Master list of all generated requests
- **fcfs_states / look_states (List[str]):** Status for each request ('pending', 'active', 'completed')
- **fcfs_wait_times / look_wait_times (List[float]):** Wait times for completed pickups
- **cpu_samples_fcfs / cpu_samples_look (List[float]):** CPU utilization samples collected every 100ms

Rationale: Using parallel lists allows independent algorithm execution while maintaining O(1) access to request information by index. The state tracking enables real-time visualization of algorithm progress.

3.2 FCFS Algorithm

Algorithm Description:

First-Come-First-Served processes requests in strict arrival order. The algorithm operates in two phases for each request:

- **Phase 1 - Move to Pickup:** Elevator moves from current position to request's starting floor, passing through all intermediate floors
- **Phase 2 - Move to Destination:** After pickup, elevator moves to destination floor, again traversing all intermediate floors

Pseudocode:

```
FOR each request in queue (in order):
```

```
    WHILE current_floor ≠ request.start:
        Move one floor closer (up or down)
        distance++
    pickup_passenger()
    record_wait_time()
    WHILE current_floor ≠ request.dest:
        Move one floor closer (up or down)
        distance++
    dropoff_passenger()
```

Time Complexity: O(n×m) where n is number of requests and m is average distance per request

Space Complexity: O(n) for storing all requests and their states

3.3 LOOK Algorithm

Algorithm Description:

LOOK (also known as the Elevator Algorithm) services requests in the current direction of travel. It moves in one direction (up or down) until no more requests exist in that direction, then reverses. At each floor, the algorithm opportunistically picks up and drops off passengers.

Pseudocode:

```
direction = STOPPED
WHILE outstanding_requests exist:
    // Handle current floor
    FOR each request at current_floor:
        IF pending: pickup_passenger()
        IF active: dropoff_passenger()
    // Determine direction
    up_requests = check_requests_above()
    down_requests = check_requests_below()
    IF direction == STOPPED:
        direction = UP if up_requests else DOWN
    ELSE IF no_requests_in_current_direction:
        direction = REVERSE
    // Move one floor
    current_floor += direction
    distance++
```

Time Complexity: O(n×m) with potential for better average case due to batching

Space Complexity: O(n) for storing requests and states, plus O(1) for direction tracking

4. KPI Results and Analysis

4.1 Performance Metrics Overview

The system measures eight key performance indicators to comprehensively evaluate algorithm performance. The following table presents typical results from a simulation with n=50 requests:

Metric	FCFS	LOOK
Execution Time (ms)	57,200	**23,800**
Memory Usage (MB)	4.54	4.54
Throughput (ops/s)	4.53	**4.25**
Avg Wait Time (s)	18.51	**0.51**
CPU Utilization (%)	**3.06**	3.14
Steps	259	**97**
Distance (floors)	159	**97**
Completed Requests	50 / 50	50 / 50

Note: Bold values indicate superior performance for that metric.

4.2 Detailed Analysis

4.2.1 Execution Time and Throughput

LOOK demonstrates 40% faster execution time (23,800ms vs. 57,200ms) due to its efficient batching strategy. By servicing multiple requests in the same direction, LOOK reduces unnecessary back-and-forth movement.

4.2.2 Distance Traveled

The most significant advantage of LOOK appears in distance traveled: 97floors compared to FCFS's 259 floors, representing a 67% reduction. This efficiency gain directly translates to energy savings and faster service. FCFS's sequential processing forces the elevator to potentially travel to opposite ends of the building for consecutive requests, while LOOK intelligently groups nearby requests.

4.2.3 Wait Time Analysis

Average wait time favors LOOK with 0.51 seconds versus FCFS's 18.51 seconds (35 times better). However, FCFS provides more predictable wait times with lower variance, as passengers know exactly when their turn will come. LOOK's wait times can vary significantly depending on whether a passenger's request aligns with the current direction of travel. A passenger requesting a downward trip while the elevator is moving upward may experience longer waits.

4.2.4 CPU Utilization

FCFS shows lower CPU utilization (3.06% vs 3.14%), reflecting its simpler decision-making process. LOOK requires additional computation to determine direction and scan for requests, with higher CPU usage. However, both values remain well below concerning levels for modern processors, making this trade-off acceptable for the efficiency gains.

4.2.5 Memory Usage

Memory usage is comparable between both algorithms (4.54MB), This similarity is expected as both algorithms store the same number of requests (O(n) space complexity). The slight increase for LOOK comes from additional direction state tracking, which has minimal impact on overall system resources.

5. Conclusions and Recommendations

5.1 Key Findings

The comparative analysis reveals distinct trade-offs between the two algorithms:

- **Efficiency vs. Simplicity:** LOOK provides 40-67% better performance in execution time, distance, and throughput at higher CPU usage and more complex implementation.
- **Fairness vs. Optimization:** FCFS guarantees predictable service order with consistent wait times, while LOOK optimizes overall system performance with variable individual wait times.
- **Resource Utilization:** Both algorithms show similar memory usage, confirming theoretical O(n) space complexity. CPU differences are negligible in absolute terms (< 5%).

5.2 Algorithm Selection Guidelines

Recommendation: Use LOOK when:

- High traffic volume requires optimization (office buildings, hotels during peak hours)
- Energy efficiency is a priority
- Total wait time matters more than individual fairness
- System can tolerate slightly higher computational overhead

Recommendation: Use FCFS when:

- Fairness and predictability are critical (government buildings, healthcare facilities)
- Low traffic volume makes optimization less impactful (residential buildings)
- System requires simple, transparent logic for maintenance
- Minimal computational resources available

5.3 Implementation Success

The project successfully achieved all stated objectives:

- Implemented two complete scheduling algorithms with accurate floor-by-floor traversal
- Created realistic simulation with proper timing (0.4s generation, 0.2s movement)
- Measured eight comprehensive KPIs with real-time tracking
- Developed intuitive GUI for visualization and comparison
- Provided data-driven analysis of algorithm trade-offs

5.4 Future Work

Several enhancements could extend this project's utility:

- **Multiple Elevators:** Implement load balancing across multiple elevator cars

- **Additional Algorithms:** Add C-SCAN, SSTF, and priority-based scheduling
- **Dynamic Traffic Patterns:** Simulate peak hours, lunch rushes, and asymmetric usage
- **Machine Learning:** Predict request patterns and optimize scheduling proactively
- **Data Export:** Generate CSV reports and statistical visualizations
- **Building Configuration:** Support variable floor counts and express zones

5.5 Final Conclusion

This project demonstrates that elevator scheduling is not a simple choice between "good" and "bad" algorithms, but rather a decision requiring careful consideration of system priorities. LOOK clearly outperforms FCFS in aggregate efficiency metrics, showing 15-27% improvements in speed and distance. However, FCFS's predictability and fairness remain valuable in contexts where individual experience matters more than system optimization.

The implementation successfully provides a framework for comparing scheduling algorithms with comprehensive metrics and visualization. The measured KPIs validate theoretical expectations while revealing practical nuances such as CPU overhead and wait time variance. This tool serves as both an educational resource for understanding scheduling algorithms and a practical testbed for evaluating different approaches.

Ultimately, the choice between FCFS and LOOK depends on specific deployment requirements, with LOOK recommended for high-traffic, efficiency-focused environments and FCFS preferred for fairness-critical, low-traffic scenarios. The project's modular design facilitates future extensions, making it a valuable foundation for continued research in elevator scheduling optimization.